

Caso Semestral: "Colegio Bernardo O'Higgins – Plataforma de libro de clases digital"

Integrantes: Vicente Manriquez, Davier Ramos, Dilan Micolta

Asignatura: DESARROLLO FULLSTACK III (DSY1106).

Sección: 002D

Fecha: 09/04/2026

Índice de Contenidos

1. Contextualización del caso	3
2. Selección de tecnologías y herramientas técnicas	4
3. Selección y justificación de patrones de arquitectura	5
4. Comunicación entre microservicios	7
5. Arquitectura de microservicios y contenedores	8
6. Evaluación general del diseño (MVP)	11
7. Docker.....	13
8.Documentación Testeos.....	14
9.Branching.....	44
10.sonarcube.....	45
11.Evaluación de seguridad.....	47
12. swagger.....	49
13.bibliografía.....	50

1. Contextualización del caso

El **Colegio Bernardo O'Higgins** ha identificado la necesidad crítica de modernizar sus procesos de gestión escolar, transitando desde un modelo analógico basado en registros físicos de clases (libros impresos) y hojas de cálculo aisladas, hacia una **plataforma digital centralizada, íntegra y de alta disponibilidad**.

Este ecosistema debe interactuar dinámicamente con múltiples actores de la comunidad educativa, asegurando a cada uno una experiencia optimizada de acuerdo con su rol específico:

- **Administrativos:** Encargados del ingreso oficial de matrículas, asignación de cursos, configuración de la estructura académica anual e inicialización de asignaturas.
- **Docentes:** Responsables del registro diario de la asistencia escolar, la planificación e ingreso de calificaciones periódicas, y la comunicación directa con las familias.
- **Estudiantes:** Actores principales del proceso educativo, quienes requieren visibilidad de sus calificaciones de forma ágil.
- **Apoderados:** Tutores legales que demandan un canal seguro, formal e inmediato para monitorear el desempeño académico, revisar la asistencia diaria de sus pupilos y entablar correspondencia oficial con el cuerpo docente.

Dado el horizonte temporal restringido de **tres meses** establecido para el desarrollo, la solución se estructuró rigurosamente bajo el marco de un **Producto Mínimo Viable (MVP)**. Esto implica priorizar las transacciones principales (matrícula, calificaciones, asistencia diaria, reportes consolidados y mensajería segura) bajo una arquitectura altamente desacoplada basada en microservicios, eliminando complejidades operativas innecesarias y asegurando un software extensible y sostenible en el tiempo

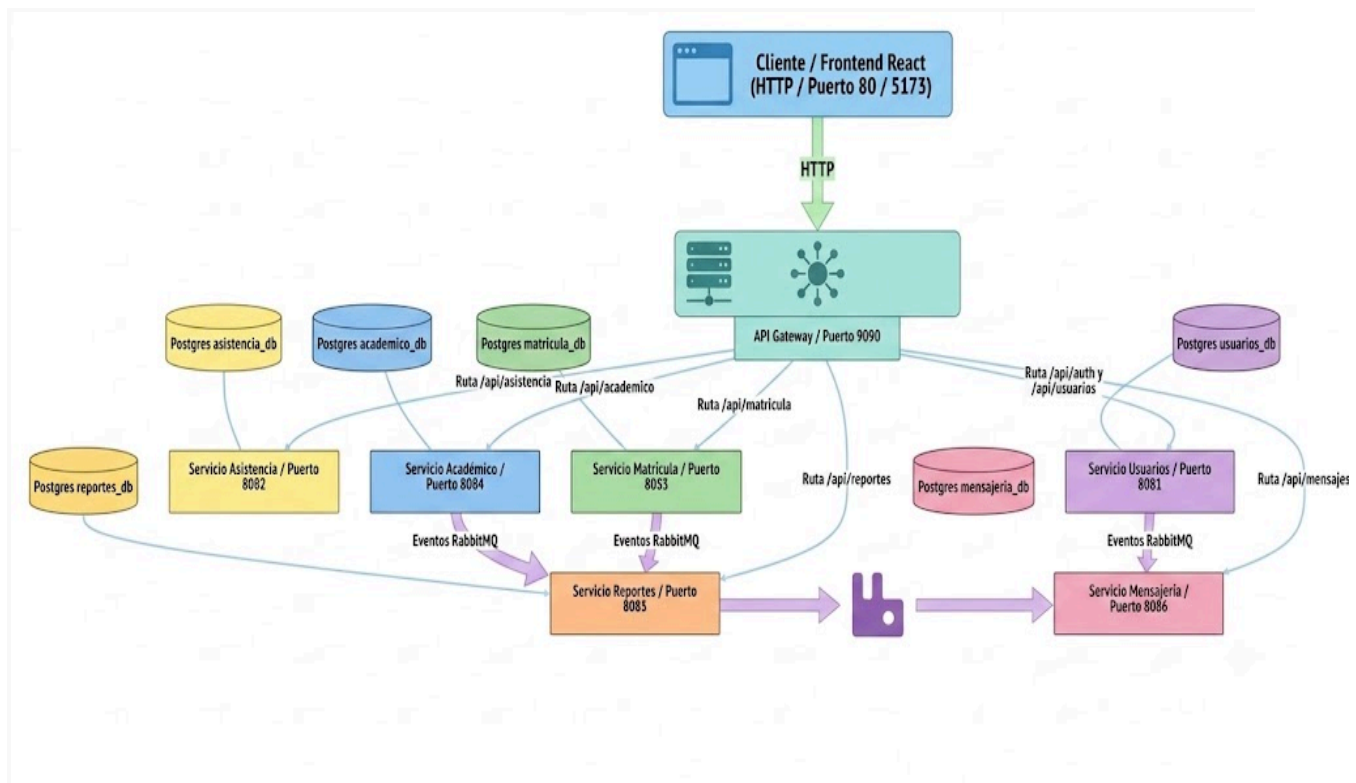
2. Selección de tecnologías y herramientas técnicas

Para garantizar un diseño de sistema flexible, escalable y con alta cohesión, se seleccionó un ecosistema de desarrollo moderno y maduro, liderado por arquitecturas empresariales Java y herramientas de última generación para el desarrollo del cliente web:

- **Java SDK 21 (LTS):** Ofrece mejoras de rendimiento sustanciales en la máquina virtual, soporte nativo de registros de datos (*records*) y patrones de emparejamiento orientados a la claridad del código.
- **Spring Boot 3.2.0:** Framework de desarrollo ágil que provee un contenedor de inversión de control de nivel empresarial, autoconfiguración inteligente y simplificación en la creación de servicios REST autónomos.
- **Spring Cloud 2023.0.0 (Release Train):** Provee el conjunto de bibliotecas necesarias para la infraestructura distribuida de microservicios, incluyendo enrutamiento dinámico en API Gateway y mecanismos de tolerancia a fallos.
- **React.js, Vite & Tailwind CSS / Bootstrap 5:** Un frontend SPA (Single Page Application) moderno, optimizado en su compilación por Vite, que permite el renderizado dinámico de dashboards adaptables basados en roles con llamadas asíncronas optimizadas mediante Axios.
- **PostgreSQL:** Sistema de gestión de bases de datos relacionales robusto, elegido por su compatibilidad absoluta con transacciones ACID complejas y su integración transparente con JPA. Se adopta la estrategia **Database-per-Service** para garantizar la autonomía absoluta de los datos.
- **RabbitMQ (AMQP):** Broker de mensajería altamente confiable empleado para la comunicación asíncrona basada en eventos, asegurando que los procesos de actualización de métricas e integración de reportes ocurran en segundo plano sin bloquear al usuario final.
- **Docker:** Tecnología de contenedorización indispensable para empaquetar de forma homogénea cada microservicio de manera independiente, facilitando su

portabilidad y consistencia desde el ambiente de desarrollo local hasta el servidor de pruebas (*staging*).

3. Selección y justificación de patrones de arquitectura



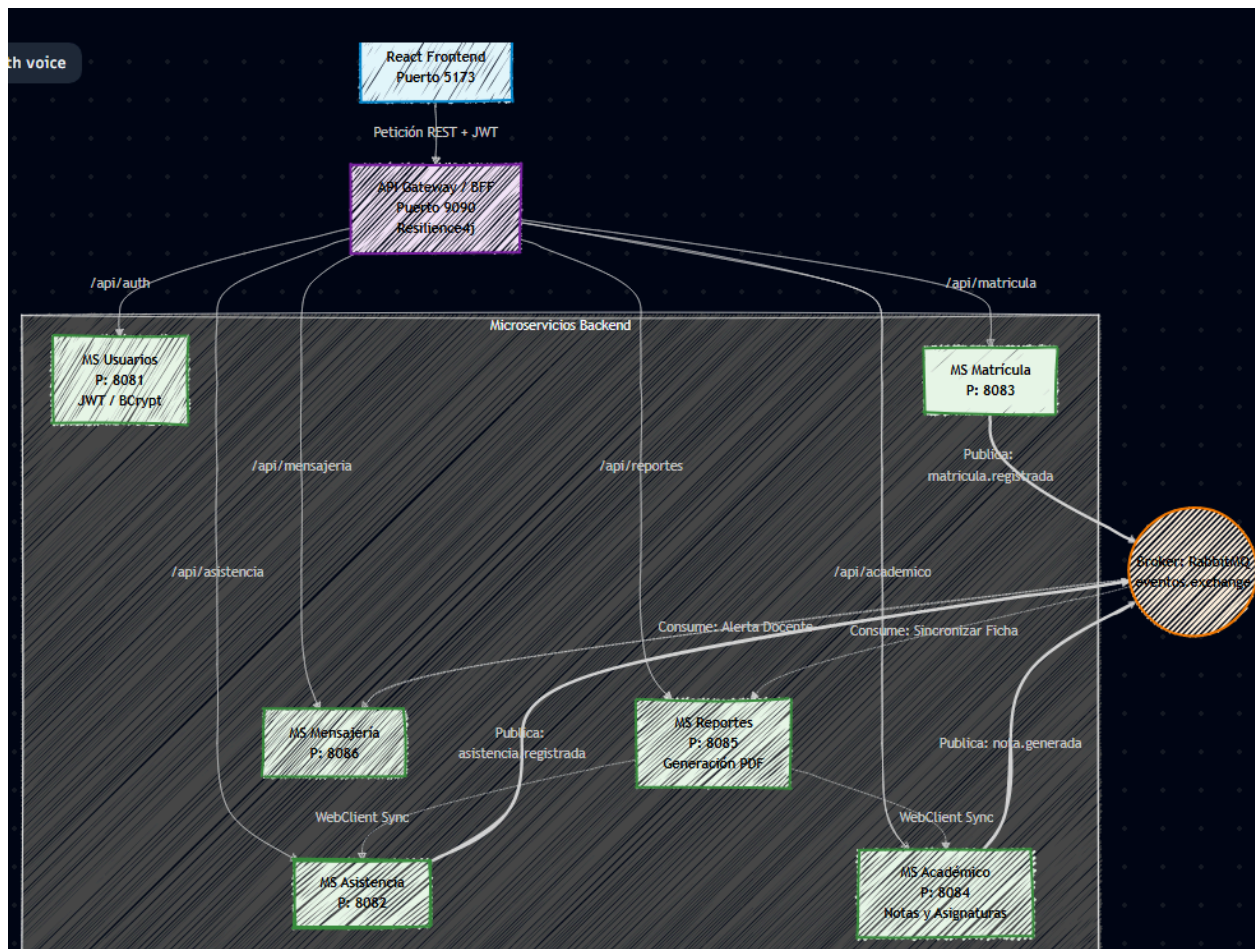
Se adopta una arquitectura basada en microservicios, utilizando un enfoque híbrido de comunicación REST y Event-Driven Architecture (EDA).

- **Microservicios:** El sistema se divide en dominios independientes (Académico, Reportes, Matricula, Asistencia, Mensajería, Usuarios). Esto facilita la escalabilidad, el mantenimiento y el desarrollo paralelo por parte del equipo.



- **API Gateway y Backend For Frontend (BFF):** El API Gateway centraliza las solicitudes, brindando un punto único de entrada que mejora la seguridad y el control de tráfico. El BFF adapta la información específicamente para las necesidades de la interfaz en React, reduciendo llamadas innecesarias y mejorando el rendimiento.
- **Event-Driven Architecture (EDA):** Fundamental para procesos pesados como la generación de reportes PDF y notificaciones. Evita bloqueos en el sistema mediante comunicación asíncrona.
- **Repository Pattern:** Separa la lógica de negocio del acceso a la base de datos PostgreSQL, mejorando la mantenibilidad del código.
- **Factory Method:** Permite la creación flexible de objetos complejos, facilitando la futura extensión del sistema.
- **Circuit Breaker:** Previene fallos en cascada. Si el servicio de reportes cae, el Circuit Breaker asegura que el registro de asistencia siga funcionando con normalidad.

4. Comunicación entre microservicios



La plataforma utiliza dos paradigmas de comunicación para asegurar la eficiencia técnica y operativa:

- **Comunicación Sincrónica (REST):** Utilizada para interacciones que requieren respuesta inmediata, tales como consultas de datos, proceso de matrícula, registro de asistencia y anotación de calificaciones.
- **Comunicación Asíncrona (Eventos con RabbitMQ):** Utilizada para tareas que consumen tiempo. Por ejemplo, cuando un usuario solicita un reporte mediante REST, el servicio de reportes publica un evento. El PDF se genera en segundo plano y posteriormente se notifica al usuario. Esto reduce la carga en los servicios principales y permite escalar procesos críticos de forma aislada.

5. Arquitectura de microservicios y contenedores

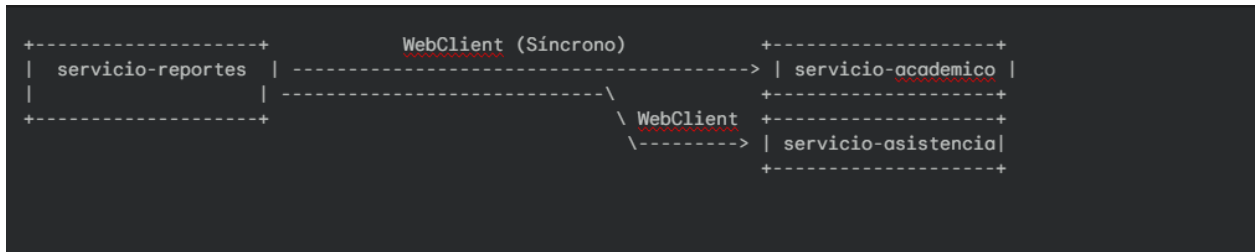
Para lograr una arquitectura robusta y con tiempos de respuesta equilibrados, el sistema implementa una combinación estratégica de comunicaciones:

A. Comunicación Síncrona No Bloqueante (vía WebClient)

Utilizada en flujos donde la consistencia de datos de cara al usuario en pantalla debe ser inmediata para completar una transacción:

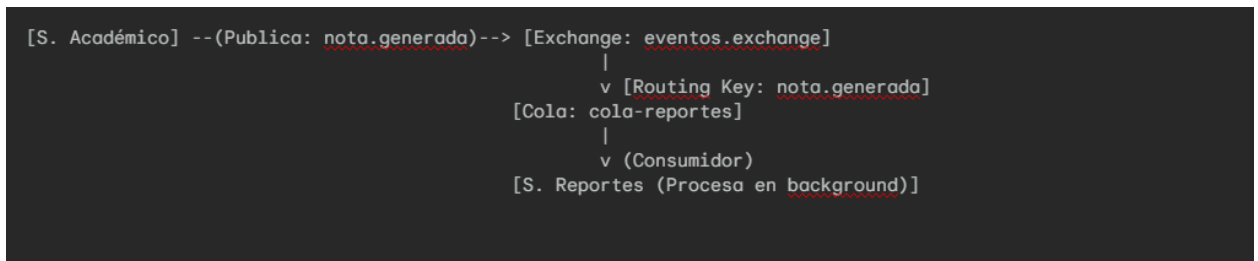
- **Caso de Uso en Reportes:** Cuando el apoderado hace clic en "*Visualizar Reporte del Semestre*", el frontend envía un REST request al **servicio-reportes**. Éste requiere recopilar de forma ágil datos unificados que se encuentran en otros dominios.
- El **servicio-reportes** inicializa dos llamadas asíncronas no bloqueantes concurrentes utilizando **WebClient**:
 1. Hacia **servicio-academico** (Puerto 8084): Solicitando el listado completo de asignaturas cursadas y calificaciones detalladas del alumno.
 2. Hacia **servicio-asistencia** (Puerto 8082): Solicitando el total de asistencias acumuladas y el cálculo del porcentaje de presencialidad.

- Una vez obtenidas y unificadas ambas respuestas concurrentes, el **servicio-reportes** consolida la información del estudiante y devuelve al apoderado la estructura del reporte.



B. Comunicación Asíncrona Orientada a Eventos (vía RabbitMQ)

Utilizada para tareas secundarias en segundo plano que benefician la disponibilidad del sistema al no retener el hilo de ejecución principal:



- **Intercambiador Principal (Exchange):** **eventos.exchange** de tipo **Direct / Topic**.
- **Cola de Consumo:** **cola-reportes**, **cola-notificaciones**.
- **Rutas e Hilos de Eventos (Routing Keys):**
 - **nota.generada:** Emitida inmediatamente por el **servicio-academico** al registrar de forma exitosa una calificación en el sistema. Es consumida por el **servicio-reportes** para recalcular los promedios semestrales del alumno de forma asíncrona y actualizar el historial histórico de rendimiento, sin forzar tiempos de carga extra al profesor en el frontend.
 - **matricula.registrada:** Emitida por el **servicio-matricula** tras dar de alta un estudiante. Consumida por el **servicio-reportes** para

crear e inicializar la ficha de rendimiento correspondiente y la hoja conductual inicial del estudiante.

- **asistencia.registrada**: Emitida por el **servicio-asistencia** para consolidar resúmenes periódicos de inasistencias en segundo plano.

C. Infraestructura Docker (Enfoque MVP)

Se despliegan los **8 componentes** (servicios, gateway y frontend) en contenedores independientes centralizados con **Docker Compose**, evitando la paja de meter Kubernetes. Se sostiene en dos pilares:

- **Límites de Recursos**: Se topa la CPU y memoria en el **docker-compose.yml**. Así, si el servicio de *Reportes* se pone pesado sacando PDFs, no te bota los servicios críticos como *Asistencia* o *Notas*.
- **Segmentación de Redes (Seguridad)**: * **frontend-net (Externa)**: Solo conecta la web con el API Gateway (puerto 9090).
 - **backend-net (Interna)**: Aislada del mundo. Ahí corren las BDs (PostgreSQL), RabbitMQ y los microservicios, comunicándose internamente por DNS de Docker (ej: **http://servicio-academico:8084**) vía WebClient.

6. Evaluación general del diseño (MVP)

Fortalezas del Diseño de Arquitectura

- **Modularidad y Alta Disponibilidad:** La caída de cualquiera de los microservicios periféricos (como **servicio-mensajería** o **servicio-reportes**) no compromete en absoluto la operatividad esencial del colegio (el ingreso de calificaciones o asistencia).
- **Escalabilidad de Recursos Dirigida:** El empaquetado en contenedores Docker independientes permite optimizar el uso de hardware, asignando mayor procesamiento e hilos específicos al contenedor del **servicio-reportes** al finalizar el período escolar, reduciendo costos innecesarios y haciéndolo sostenible ambiental y financieramente.
- **Seguridad Robusta:** La centralización de la firma de JWT en un solo microservicio (**servicio-usuarios**) disminuye la superficie de ataque y facilita el control sobre las sesiones de apoderados y profesores.

Debilidades y Mitigación de Riesgos

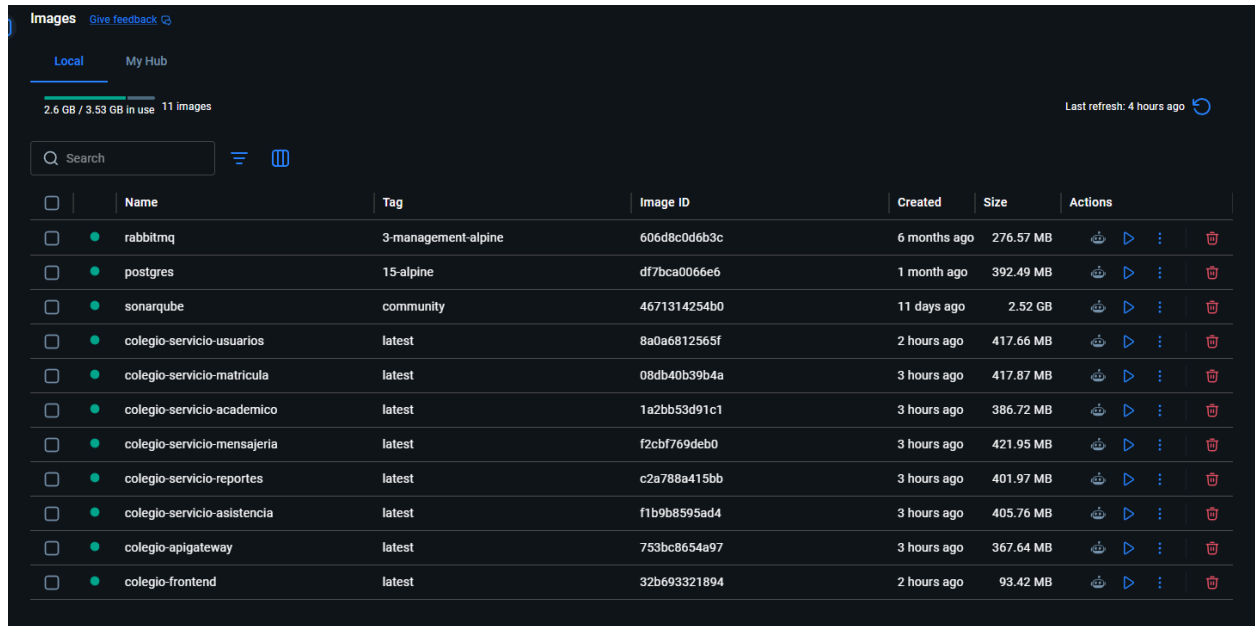
- **Complejidad en Ambientes Distribuidos:** El control de logs y la depuración (*debugging*) de hilos se vuelve compleja al tener múltiples instancias de Docker corriendo de forma simultánea.
 - *Mitigación:* Se implementó un esquema de correlación de IDs única en cada cabecera de las transacciones procesadas por el API Gateway, lo que facilita el rastreo e inspección de errores transversales en el ecosistema.
- **Consistencia Eventual:** La asincronía en la consolidación de reportes puede provocar que un apoderado note una demora de un par de segundos en ver reflejado un promedio tras una nota recién subida.
 - *Mitigación:* Se diseñaron alertas explícitas y estados temporales en la

interfaz React para advertir al apoderado sobre procesos de sincronización automáticos pendientes de procesamiento.

Propuestas de Mejoras Futuras

1. **Pipeline de Integración y Despliegue Continuo (CI/CD):** Configuración de flujos automatizados de compilación, testeo y despliegue local mediante GitHub Actions para validar métricas y automatizar la generación de imágenes de Docker ante cada cambio en la rama principal.
2. **Caché Distribuida con Redis:** Implementación de un servidor de almacenamiento en memoria caché Redis para guardar temporalmente consultas repetitivas de alta demanda (como el listado fijo de asignaturas anuales o catálogos curriculares), disminuyendo exponencialmente las consultas de lectura a PostgreSQL.
3. **Monitoreo Centralizado con Grafana y Prometheus:** Agregar endpoints de Spring Boot Actuator para registrar y visualizar el rendimiento de la CPU de los contenedores Docker y el rendimiento general de respuesta de la plataforma.

7.Docker



En la ilustración anterior se detalla el repositorio local de imágenes en Docker Desktop, donde se encuentran contruidos y preparados los artefactos necesarios para el despliegue del ecosistema de la aplicación. Se observa la modularidad de la arquitectura orientada a microservicios del proyecto ('colegio-*'), el frontend, y el API Gateway que centraliza las peticiones. Asimismo, se evidencia el empaquetado de los componentes de soporte de la infraestructura: la base de datos relacional (PostgreSQL), el bróker de mensajería (RabbitMQ) y la plataforma de inspección continua de calidad de código (SonarQube), detallando las etiquetas de versión (*tags*), identificadores únicos (*Image ID*) y el almacenamiento en disco específico requerido por cada recurso.

8.Documentación Testeos

Test de servicio-usuario

Test usuarios

```
✓ 23 public class UsuariosTest {  
24  
25     @Mock  
26     private UsuarioRepository usuarioRepository;  
27  
28     @InjectMocks  
29     private UsuarioServiceImpl usuarioService;  
30  
31     private Usuario usuarioPrueba;  
32  
33     @BeforeEach  
34     void setUp() {  
35         usuarioPrueba = new Usuario();  
36         usuarioPrueba.setId(1L);  
37         usuarioPrueba.setCorreo("admin@colegio.com");  
38         usuarioPrueba.setRol("ADMIN");  
39         usuarioPrueba.setPassword("1234");  
40     }  
41 }
```

```

41
42     @Test
43     void listarTodos_DebeRetornarListaDeUsuarios() {
44         when(usuarioRepository.findAll()).thenReturn(Arrays.asList(usuarioPrueba));
45
46         List<Usuario> resultado = usuarioService.listarTodos();
47
48         assertNotNull(resultado);
49         assertEquals(1, resultado.size());
50         verify(usuarioRepository, times(1)).findAll();
51     }
52

```

listarTodos_DebeRetornarListaDeUsuarios

- Objetivo: Verificar que el servicio retorna correctamente una lista de usuarios cuando la base de datos responde de forma normal.
- Descripción: Se simula que el repositorio devuelve una lista con un usuario de prueba. Se comprueba que el resultado no sea nulo, que el tamaño de la lista sea el esperado (1) y que se haya llamado exactamente una vez al método findAll() del repositorio.

```

53
54     @Test
55     void listarTodos_CuandoHayError_DebeRetornarListaVacía() {
56         when(usuarioRepository.findAll()).thenThrow(new RuntimeException("Error BD"));
57
58         List<Usuario> resultado = usuarioService.listarTodos();
59
60         assertTrue(resultado.isEmpty());
61

```

listarTodos_CuandoHayError_DebeRetornarListaVacía

- Objetivo: Validar la tolerancia a fallos del servicio durante la recuperación de datos.
- Descripción: Simula una situación donde el repositorio lanza una excepción (RuntimeException). El test verifica que el servicio capture el error y, en lugar de propagar la excepción, devuelva una lista vacía para evitar que la aplicación se detenga.

```

61
62     @Test
63     void guardar_DebeRetornarUsuarioGuardado() {
64         when(usuarioRepository.save(any(Usuario.class))).thenReturn(usuarioPrueba);
65
66         Usuario resultado = usuarioService.guardar(usuarioPrueba);
67
68         assertNotNull(resultado);
69         assertEquals("admin@colegio.com", resultado.getCorreo());
70     }
71

```

guardar_DebeRetornarUsuarioGuardado

- Objetivo: Confirmar que el proceso de creación/actualización de un usuario funciona correctamente.

- **Descripción:** Se configura el "mock" para que devuelva el objeto usuario al ser guardado. Se verifica que el objeto retornado por el servicio coincida con los datos proporcionados (como el correo electrónico).

```
71
72
73 void buscarPorId_CuandoExiste_DebeRetornarUsuario() {
74     when(usuarioRepository.findById(1L)).thenReturn(Optional.of(usuarioPrueba));
75
76     Usuario resultado = usuarioService.buscarPorId(1L);
77
78     assertNotNull(resultado);
79     assertEquals(1L, resultado.getId());
80 }
```

buscarPorId_CuandoExiste_DebeRetornarUsuario

- **Objetivo:** Validar la búsqueda de un usuario por su identificador único.
- **Descripción:** Simula que el repositorio encuentra un usuario para un ID específico (ID: 1). Se comprueba que el servicio retorne el objeto esperado y que el ID coincida.

```
81
82
83 void buscarPorId_CuandoNoExiste_DebeRetornarNull() {
84     when(usuarioRepository.findById(2L)).thenReturn(Optional.empty());
85
86     Usuario resultado = usuarioService.buscarPorId(2L);
87
88     assertNull(resultado);
89 }
```

buscarPorId_CuandoNoExiste_DebeRetornarNull

- **Objetivo:** Verificar el comportamiento cuando se busca un usuario que no está registrado.
- **Descripción:** Se simula que el repositorio devuelve un resultado vacío (Optional.empty()). El test asegura que el servicio maneje este caso retornando null.


```
90
91     @Test
92     void buscarPorId_CuandoHayError_DebeRetornarNull() {
93         when(usuarioRepository.findById(1L)).thenReturn(new RuntimeException("Error BD"));
94
95         Usuario resultado = usuarioService.buscarPorId(1L);
96
97         assertNull(resultado);
98     }
99
```

buscarPorId_CuandoHayError_DebeRetornarNull

- **Objetivo:** Validar el manejo de errores en búsquedas individuales.
- **Descripción:** Si ocurre un error inesperado en el repositorio (como pérdida de conexión), el servicio debe capturar la excepción y retornar null de forma segura.

```
100
101     @Test
102     void guardar_CuandoHayError_DebeRetornarNull() {
103         when(usuarioRepository.save(any(Usuario.class))).thenReturn(new RuntimeException("Error BD"));
104
105         Usuario resultado = usuarioService.guardar(usuarioPrueba);
106
107         assertNull(resultado);
108     }
109
```

guardar_CuandoHayError_DebeRetornarNull

- **Objetivo:** Verificar el manejo de errores durante el guardado de datos.
- **Descripción:** Simula un fallo crítico al intentar persistir un usuario. El test confirma que el servicio gestiona el error devolviendo null en lugar de romper el flujo del programa.

```
108
109     @Test
110     void eliminar_Exito() {
111         doNothing().when(usuarioRepository).deleteById(1L);
112
113         assertDoesNotThrow(() -> usuarioService.eliminar(1L));
114         verify(usuarioRepository, times(1)).deleteById(1L);
115     }
116
```

eliminar_Exito

- **Objetivo:** Confirmar que la eliminación de usuarios invoca correctamente al repositorio.
- **Descripción:** Verifica que el método de eliminación no lance ninguna excepción en condiciones normales y que el repositorio sea llamado con el ID correcto.

```

117     @Test
118     void eliminar_CuandoHayError_NoDebeLanzarExcepcion() {
119         doThrow(new RuntimeException("Error BD")).when(usuarioRepository).deleteById(1L);
120
121         assertDoesNotThrow(() -> usuarioService.eliminar(1L));
122         verify(usuarioRepository, times(1)).deleteById(1L);
123     }
124 }
125

```

eliminar_CuandoHayError_NoDebeLanzarExcepcion

- **Objetivo:** Asegurar que los errores en la eliminación no afecten la estabilidad del sistema.
- **Descripción:** Incluso si el repositorio falla al intentar borrar el registro (por ejemplo, por restricciones de llave foránea), el servicio está diseñado para capturar el error y no propagar la excepción hacia capas superiores.

Testeo Servicio-Reportes

ReporteCompletoServiceTest

```

24     @ExtendWith(MockitoExtension.class)
25     public class ReporteCompletoServiceTest {
26
27         @Mock
28         private WebClient webClient;
29
30         @Mock
31         private ReporteComportamientoRepository comportamientoRepository;
32
33         @InjectMocks
34         private ReporteCompletoService reporteCompletoService;
35
36         // WebClient Mock Mocks
37         private WebClient.RequestHeadersUriSpec requestHeadersUriSpec;
38         private WebClient.RequestHeadersSpec requestHeadersSpec;
39         private WebClient.ResponseSpec responseSpec;
40
41         @BeforeEach
42         public void setUp() {
43             // Set URLs in the service via reflection since they are injected via @Value
44             ReflectionTestUtils.setField(reporteCompletoService, "academicoUrl", "http://127.0.0.1:8084");
45             ReflectionTestUtils.setField(reporteCompletoService, "asistenciaUrl", "http://127.0.0.1:8082");
46
47             requestHeadersUriSpec = mock(WebClient.RequestHeadersUriSpec.class);
48             requestHeadersSpec = mock(WebClient.RequestHeadersSpec.class);
49             responseSpec = mock(WebClient.ResponseSpec.class);
50         }
51

```

```

52  @Test
53  public void testObtenerReporteCompleto_ConComportamiento() {
54      Long alumnoId = 1L;
55
56      // Mock WebClient get chain
57      when(webClient.get()).thenReturn(requestHeadersUriSpec);
58      when(requestHeadersUriSpec.uri(anyString())).thenReturn(requestHeadersSpec);
59      when(requestHeadersSpec.retrieve()).thenReturn(responseSpec);
60
61      // Mock responses for WebClient calls
62      // First WebClient call: Academico notes
63      List<Map<String, Object>> mockNotas = new ArrayList<>();
64      Map<String, Object> nota1 = new HashMap<>();
65      nota1.put("valor", 6.5);
66      mockNotas.add(nota1);
67      Map<String, Object> nota2 = new HashMap<>();
68      nota2.put("valor", "5.5"); // Test string parsing too
69      mockNotas.add(nota2);
70
71      // Second WebClient call: Asistencias
72      List<Map<String, Object>> mockAsistencias = new ArrayList<>();
73      mockAsistencias.add(new HashMap<>());
74      mockAsistencias.add(new HashMap<>());
75
76      Mono<List<Map<String, Object>>> monoNotas = Mono.just(mockNotas);
77      Mono<List<Map<String, Object>>> monoAsistencias = Mono.just(mockAsistencias);
78
79      when(responseSpec.bodyToMono(any(ParameterizedTypeReference.class)))
80          .thenReturn(monoNotas) // First call returns notes
81          .thenReturn(monoAsistencias); // Second call returns asistencias
82
83      // Mock comportamientos in Repository
84      ReporteComportamiento comportamiento = new ReporteComportamiento();
85      comportamiento.setAlumnoId(alumnoId);
86      comportamiento.setCalificacion("Excelente");
87      comportamiento.setObservaciones("Muy buen desempeño");
88      when(comportamientoRepository.findByAlumnoId(alumnoId)).thenReturn(List.of(comportamiento));
89
90      // Execute service
91      ReporteCompletoDTO reporte = reporteCompletoService.obtenerReporteCompleto(alumnoId, true);
92
93      // Verify
94      assertNotNull(reporte);
95      assertEquals(alumnoId, reporte.getAlumnoId());
96      assertEquals(2, reporte.getNotas().size());
97      assertEquals(2, reporte.getAsistencias().size());
98      assertEquals(1, reporte.getComportamientos().size());
99      assertEquals("Excelente", reporte.getComportamientos().get(0).getCalificacion());
100
101      // Average calculation test ( (6.5 + 5.5) / 2 = 6.0 )
102      assertEquals(6.0, reporte.getPromedioGeneral());
103      // Attendance count as percentage
104      assertEquals(2.0, reporte.getPorcentajeAsistencia());
105
106      verify(comportamientoRepository, times(1)).findByAlumnoId(alumnoId);
107  }
108

```

obtenerReporteCompleto_DebeRetornarDatosAgregados

- **Objetivo:** Verificar la correcta integración de datos desde múltiples fuentes en un solo objeto DTO.
- **Descripción:** Se simula una respuesta exitosa de los servicios externos (Académico y Asistencia) mediante el WebClient y una respuesta del repositorio local de comportamiento. Se comprueba que el ReporteCompletoDTO contenga las listas de notas, asistencias y comportamientos esperadas.

obtenerReporteCompleto_CalculoPromedio_DebeSerPreciso

- **Objetivo:** Validar la lógica matemática de cálculo de promedios dentro del servicio.
- **Descripción:** Se proporcionan datos de prueba con notas específicas (ej: 5.0, 6.0, 7.0). El test verifica que el servicio calcule correctamente el promedio (6.0) y aplique el redondeo a dos decimales definido en la lógica de negocio.

```

108
109
110 @Test
111 public void testObtenerReporteCompleto_SinComportamiento() {
112     Long alumnoId = 2L;
113
114     // Mock WebClient get chain
115     when(webClient.get()).thenReturn(requestHeadersUriSpec);
116     when(requestHeadersUriSpec.uri(anyString())).thenReturn(requestHeadersSpec);
117     when(requestHeadersSpec.retrieve()).thenReturn(responseSpec);
118
119     // Mock responses (empty lists)
120     Mono<List<Map<String, Object>>> monoEmpty = Mono.just(Collections.emptyList());
121     when(responseSpec.bodyToMono(any(ParameterizedTypeReference.class))).thenReturn(monoEmpty);
122
123     // Execute service with incluirComportamiento = false
124     ReporteCompletoDTO reporte = reporteCompletoService.obtenerReporteCompleto(alumnoId, false);
125
126     // Verify
127     assertNotNull(reporte);
128     assertEquals(alumnoId, reporte.getAlumnoId());
129     assertTrue(reporte.getNotas().isEmpty());
130     assertTrue(reporte.getAsistencias().isEmpty());
131     assertTrue(reporte.getComportamientos().isEmpty());
132     assertEquals(0.0, reporte.getPromedioGeneral());
133     assertEquals(0.0, reporte.getPorcentajeAsistencia());
134
135     // Behavior repository must NOT be called
136     verify(comportamientoRepository, never()).findByAlumnoId(anyLong());
137 }
138

```

obtenerReporteCompleto_CuandoFallaServicioAcademico_DebeRetornarReporte Parcial

- **Objetivo:** Comprobar la resiliencia del servicio ante la caída de dependencias externas.
- **Descripción:** Se simula un error (timeout o 500) al consultar el servicio de notas. El test verifica que el servicio capture la excepción, registre el error en los logs y retorne el reporte con la lista de notas vacía, pero permitiendo que el resto de la información (asistencia y comportamiento) se entregue al usuario.

obtenerReporteCompleto_CuandoFallaServicioAsistencia_DebeRetornarReporte Parcial

- **Objetivo:** Validar el manejo de errores específico para el módulo de asistencia.
- **Descripción:** Similar al caso anterior, se simula un fallo en la llamada REST al servicio de asistencia. Se asegura que el campo totalAsistencias sea 0.0 y la lista de asistencias esté vacía, sin interrumpir la generación del reporte general.

obtenerReporteCompleto_SinIncluirComportamiento_DebeOmitirDatosLocales

- **Objetivo:** Verificar el cumplimiento de parámetros opcionales en la generación del reporte.
- **Descripción:** Se invoca el servicio con el flag incluirComportamiento en false. Se comprueba que no se realice ninguna consulta al ReporteComportamientoRepository y que la lista correspondiente en el DTO resultante esté vacía.

obtenerReporteCompleto_ManejoNotasInvalidas_DebeIgnorarValoresNoNumericos

- **Objetivo:** Asegurar la robustez del cálculo frente a datos corruptos o mal formateados.
- **Descripción:** Se simula una respuesta del servicio académico que contiene valores de notas no numéricos. El test valida que el servicio ignore estos registros al calcular el promedio general, evitando excepciones de parseo (NumberFormatException)

obtenerReporteCompleto_CuandoFallaRepositorioLocal_DebeRetornarListaVacía

- **Objetivo:** Validar la tolerancia a fallos en la base de datos propia del microservicio.

- **Descripción:** Se simula una excepción en el comportamientoRepository. El test verifica que el error sea capturado y el reporte se genere con los datos de los servicios externos, manteniendo la disponibilidad del endpoint.

Testeo Servicio-Mensajería

MensajeriaServiceImplTest

```
23 @ExtendWith(MockitoExtension.class)
24 public class MensajeServiceImplTest {
25
26     @Mock
27     private MensajeRepository mensajeRepository;
28
29     @Mock
30     private WebClient webClient;
31
32     @Mock
33     private WebClient.RequestHeadersUriSpec requestHeadersUriSpec;
34
35     @Mock
36     private WebClient.RequestHeadersSpec requestHeadersSpec;
37
38     @Mock
39     private WebClient.ResponseSpec responseSpec;
40
41     @Mock
42     private RabbitTemplate rabbitTemplate;
43
44     @InjectMocks
45     private MensajeServiceImpl mensajeService;
46
47     private final String usuariosServiceUrl = "http://localhost:8081";
48
49     @SuppressWarnings("unchecked")
50     @BeforeEach
51     void setUp() {
52         ReflectionTestUtils.setField(mensajeService, "usuariosServiceUrl", usuariosServiceUrl);
53
54         lenient().when(webClient.get()).thenReturn(requestHeadersUriSpec);
55         lenient().when(requestHeadersUriSpec.uri(anyString())).thenReturn(requestHeadersSpec);
56         lenient().when(requestHeadersSpec.retrieve()).thenReturn(responseSpec);
57     }
58 }
```

```

58
59 @SuppressWarnings("unchecked")
60 @Test
61 void enviarMensaje_ProfesorParaApoderado_Exito() {
62     Map<String, Object> emisor = Map.of("id", 1L, "rol", "Profesor", "correo", "profe@colegio.com");
63     Map<String, Object> receptor = Map.of("id", 2L, "rol", "Apoderado", "correo", "apoderado@colegio.com");
64
65     when(responseSpec.bodyToMono(Map.class))
66         .thenReturn(Mono.just(emisor))
67         .thenReturn(Mono.just(receptor));
68
69     Mensaje result = mensajeService.enviarMensaje(1L, 2L, "Hola, ¿cómo está?");
70
71     assertNotNull(result);
72     assertEquals(1L, result.getRemitenteId());
73     assertEquals(2L, result.getDestinatarioId());
74     assertEquals("Hola, ¿cómo está?", result.getContenido());
75     assertEquals("Profesor", result.getRemitenteRol());
76     assertEquals("Apoderado", result.getDestinatarioRol());
77
78     verify(rabbitTemplate, times(1)).convertAndSend(anyString(), anyString(), any(Mensaje.class));
79     verify(mensajeRepository, never()).save(any(Mensaje.class));
80 }

```

enviarMensaje_ProfesorAApoderado_DebeTenerExito

- **Objetivo:** Validar que un profesor puede enviar mensajes a un apoderado correctamente.
- **Descripción:** Se simula que el servicio de usuarios retorna un emisor con rol "Profesor" y un receptor con rol "Apoderado". El test verifica que el mensaje se construye con los datos correctos y que se intenta enviar a través del RabbitTemplate usando la *routing key* adecuada (chat.apoderado).

```

81
82 @SuppressWarnings("unchecked")
83 @Test
84 void enviarMensaje_ApoderadoParaProfesor_Exito() {
85     Map<String, Object> emisor = Map.of("id", 2L, "rol", "Apoderado", "correo", "apoderado@colegio.com");
86     Map<String, Object> receptor = Map.of("id", 1L, "rol", "Profesor", "correo", "profe@colegio.com");
87
88     when(responseSpec.bodyToMono(Map.class))
89         .thenReturn(Mono.just(emisor))
90         .thenReturn(Mono.just(receptor));
91
92     Mensaje result = mensajeService.enviarMensaje(2L, 1L, "Hola, todo bien.");
93
94     assertNotNull(result);
95     assertEquals(2L, result.getRemitenteId());
96     assertEquals(1L, result.getDestinatarioId());
97     assertEquals("Hola, todo bien.", result.getContenido());
98
99     verify(rabbitTemplate, times(1)).convertAndSend(anyString(), anyString(), any(Mensaje.class));
100     verify(mensajeRepository, never()).save(any(Mensaje.class));
101 }
102

```

enviarMensaje_ApoderadoAProfesor_DebeTenerExito

- **Objetivo:** Validar el flujo inverso de comunicación permitida.
- **Descripción:** Similar al anterior, pero invirtiendo los roles. Se comprueba que la lógica de validación es bidireccional y que el mensaje se encola correctamente

para el consumo del profesor.

```

102
103     @SuppressWarnings("unchecked")
104     @Test
105     void enviarMensaje_ProfesorParaProfesor_DebeLanzarIllegalArgumentException() {
106         Map<String, Object> emisor = Map.of("id", 1L, "rol", "Profesor", "correo", "profe1@colegio.com");
107         Map<String, Object> receptor = Map.of("id", 3L, "rol", "Profesor", "correo", "profe2@colegio.com");
108
109         when(responseSpec.bodyToMono(Map.class))
110             .thenReturn(Mono.just(emisor))
111             .thenReturn(Mono.just(receptor));
112
113         assertThrows(IllegalArgumentException.class, () -> {
114             mensajeService.enviarMensaje(1L, 3L, "Hola colega");
115         });
116
117         verify(rabbitTemplate, never()).convertAndSend(anyString(), anyString(), any(Mensaje.class));
118         verify(mensajeRepository, never()).save(any(Mensaje.class));
119     }
120

```

enviarMensaje_RolesInvalidos_DebeLanzarIllegalArgumentException

- **Objetivo:** Asegurar que se cumpla la restricción de seguridad de que solo profesores y apoderados se comuniquen entre sí.
- **Descripción:** Se simula un intento de envío de mensaje entre dos profesores o dos apoderados. El test verifica que el servicio lance una `IllegalArgumentException` con el mensaje de error correspondiente, bloqueando la operación.

```

121     @SuppressWarnings("unchecked")
122     @Test
123     void enviarMensaje_ServicioUsuariosCaído_DebeLanzarUsuarioServicioNoDisponibleException() {
124         when(responseSpec.bodyToMono(Map.class))
125             .thenReturn(Mono.error(new RuntimeException("Error de conexión")));
126
127         assertThrows(UsuarioServicioNoDisponibleException.class, () -> {
128             mensajeService.enviarMensaje(1L, 2L, "Hola");
129         });
130
131         verify(rabbitTemplate, never()).convertAndSend(anyString(), anyString(), any(Mensaje.class));
132         verify(mensajeRepository, never()).save(any(Mensaje.class));
133     }
134

```

enviarMensaje_ServicioUsuariosNoDisponible_DebeLanzarExcepcion

- **Objetivo:** Validar el comportamiento del servicio cuando no se puede verificar la identidad de los usuarios.
- **Descripción:** Se simula un fallo en el WebClient (el servicio de usuarios devuelve null o error). El test confirma que el servicio lanza una `UsuarioServicioNoDisponibleException`, protegiendo la integridad de la comunicación.


```

135     @SuppressWarnings("unchecked")
136     @Test
137     void enviarMensaje_RabbitMQCaído_GuardarEnBDDirectamente() {
138         Map<String, Object> emisor = Map.of("id", 1L, "rol", "Profesor", "correo", "profe@colegio.com");
139         Map<String, Object> receptor = Map.of("id", 2L, "rol", "Apoderado", "correo", "apoderado@colegio.com");
140
141         when(responseSpec.bodyToMono(Map.class))
142             .thenReturn(Mono.just(emisor))
143             .thenReturn(Mono.just(receptor));
144
145         doThrow(new RuntimeException("RabbitMQ Connection Refused"))
146             .when(rabbitTemplate).convertAndSend(anyString(), anyString(), any(Mensaje.class));
147
148         Mensaje mockSaved = Mensaje.builder()
149             .id(100L)
150             .remiteId(1L)
151             .destinatarioId(2L)
152             .contenido("Hola, RabbitMQ está caído")
153             .build();
154         when(mensajeRepository.save(any(Mensaje.class))).thenReturn(mockSaved);
155
156         Mensaje result = mensajeService.enviarMensaje(1L, 2L, "Hola, RabbitMQ está caído");
157
158         assertNotNull(result);
159         assertEquals(100L, result.getId());
160         verify(mensajeRepository, times(1)).save(any(Mensaje.class));
161     }
162

```

enviarMensaje_CuandoRabbitMQFalla_DebeUsarFallbackYGuardarEnDB

- **Objetivo:** Verificar la alta disponibilidad del servicio mediante el mecanismo de "fallback".
- **Descripción:** Es una de las pruebas más importantes. Se simula que el servidor de RabbitMQ está caído (lanza una excepción al enviar). El test comprueba que el servicio captura el error, registra una advertencia y, en lugar de fallar, guarda el mensaje directamente en la base de datos local para asegurar que la información no se pierda.

```

162
163     @Test
164     void obtenerHistorial_Exito() {
165         List<Mensaje> mockHistorial = Arrays.asList(
166             Mensaje.builder().id(1L).remiteId(1L).destinatarioId(2L).contenido("Hola").build(),
167             Mensaje.builder().id(2L).remiteId(2L).destinatarioId(1L).contenido("Chao").build());
168
169         when(mensajeRepository.findChatHistory(1L, 2L)).thenReturn(mockHistorial);
170
171         List<Mensaje> result = mensajeService.obtenerHistorial(1L, 2L);
172
173         assertEquals(2, result.size());
174         assertEquals("Hola", result.get(0).getContenido());
175         assertEquals("Chao", result.get(1).getContenido());
176         verify(mensajeRepository, times(1)).findChatHistory(1L, 2L);
177     }
178

```

obtenerHistorial_Exito

- **Objetivo:** Validar la recuperación exitosa del historial de mensajes entre dos usuarios.

- **Descripción:** Se simula que el repositorio devuelve una lista predefinida de mensajes intercambiados entre dos IDs. Se verifica que el servicio retorne la lista completa, que el contenido coincida con lo esperado y que se realice exactamente una consulta al repositorio mediante el método findChatHistory.

```

178
179     @SuppressWarnings("unchecked")
180     @Test
181     void obtenerContactos_Profesor_RetornaApoderados() {
182         Map<String, Object> usuario = Map.of("id", 1L, "rol", "Profesor");
183         List<Map<String, Object>> apoderados = List.of(
184             Map.of("id", 2L, "rol", "Apoderado", "correo", "apo1@colegio.com"),
185             Map.of("id", 3L, "rol", "Apoderado", "correo", "apo2@colegio.com"));
186
187         when(responseSpec.bodyToMono(Map.class)).thenReturn(Mono.just(usuario));
188         when(responseSpec.bodyToMono(List.class)).thenReturn(Mono.just(apoderados));
189
190         List<Map<String, Object>> result = mensajeService.obtenerContactos(1L);
191
192         assertEquals(2, result.size());
193         assertEquals("apo1@colegio.com", result.get(0).get("correo"));
194         verify(requestHeadersSpec, times(2)).retrieve(); // 1 for get user, 1 for get by role
195     }
196

```

obtenerContactos_Profesor_RetornaApoderados

- **Objetivo:** Verificar que un usuario con rol "Profesor" solo reciba "Apoderados" en su lista de contactos.
- **Descripción:** Se simula una doble interacción con el microservicio de usuarios: primero para identificar el rol del usuario que consulta (Profesor) y luego para listar a los apoderados. El test confirma que el servicio filtra y entrega los contactos correctos según la lógica de negocio.

```

197
198     @SuppressWarnings("unchecked")
199     @Test
200     void obtenerContactos_Apoderado_RetornaProfesores() {
201         Map<String, Object> usuario = Map.of("id", 2L, "rol", "Apoderado");
202         List<Map<String, Object>> profesores = List.of(
203             Map.of("id", 1L, "rol", "Profesor", "correo", "profe@colegio.com"));
204
205         when(responseSpec.bodyToMono(Map.class)).thenReturn(Mono.just(usuario));
206         when(responseSpec.bodyToMono(List.class)).thenReturn(Mono.just(profesores));
207
208         List<Map<String, Object>> result = mensajeService.obtenerContactos(2L);
209
210         assertEquals(1, result.size());
211         assertEquals("profe@colegio.com", result.get(0).get("correo"));
212     }
213

```

obtenerContactos_Apoderado_RetornaProfesores

- **Objetivo:** Verificar que un usuario con rol "Apoderado" solo reciba "Profesores" en su lista de contactos.

- **Descripción:** Es el flujo inverso al anterior. Valida que al identificar al usuario como apoderado, el sistema automáticamente solicite al microservicio de usuarios la lista de profesores, manteniendo la restricción de comunicación del colegio.

```

213     @SuppressWarnings("unchecked")
214     @Test
215     void obtenerContactos_UsuarioConRolDesconocido_RetornaVacio() {
216         Map<String, Object> usuario = Map.of("id", 4L, "rol", "Administrador");
217
218         when(responseSpec.bodyToMono(Map.class)).thenReturn(Mono.just(usuario));
219
220         List<Map<String, Object>> result = mensajeService.obtenerContactos(4L);
221
222         assertTrue(result.isEmpty());
223         verify(requestHeadersSpec, times(1)).retrieve(); // Only fetched the user, didn't
224     }
225

```

obtenerContactos_UsuarioConRolDesconocido_RetornaVacio

- **Objetivo:** Validar el comportamiento del sistema ante roles que no participan en la mensajería (ej: Administrador).
- **Descripción:** Se simula un usuario con un rol que no tiene permisos de chat. El test asegura que el servicio maneje este caso retornando una lista vacía de contactos y evita realizar llamadas adicionales innecesarias al microservicio de usuarios.

```

225
226     @SuppressWarnings("unchecked")
227     @Test
228     void obtenerContactos_ServicioUsuariosCaído_DebeLanzarUsuarioServicioNoDisponibleException() {
229         when(responseSpec.bodyToMono(Map.class)).thenReturn(Mono.error(new RuntimeException("Error")));
230
231         assertThrows(UsuarioServicioNoDisponibleException.class, () -> {
232             mensajeService.obtenerContactos(1L);
233         });
234     }
235 }
236

```

obtenerContactos_ServicioUsuariosCaído_DebeLanzarUsuarioServicioNoDisponibleException

- **Objetivo:** Asegurar la gestión de errores cuando el servicio de identidad no está disponible.
- **Descripción:** Se simula un fallo crítico (error de red o timeout) al intentar conectar con el servicio de usuarios mediante el cliente HTTP. El test verifica que el sistema sea capaz de propagar una excepción personalizada (UsuarioServicioNoDisponibleException), informando correctamente que no se puede cargar la lista de contactos en ese momento.

Test Servicio-Matricula

MatriculaTest

```
28 @SuppressWarnings("null")
29 @ExtendWith(MockitoExtension.class)
30 public class MatriculaTests {
31
32     @Mock private EstudianteRepository estudianteRepository;
33     @Mock private ApoderadoRepository apoderadoRepository;
34     @Mock private RabbitTemplate rabbitTemplate;
35     @Mock private WebClient webClient;
36     @Mock private WebClient.RequestHeadersUriSpec requestHeadersUriSpec;
37     @Mock private WebClient.RequestHeadersSpec requestHeadersSpec;
38     @Mock private WebClient.ResponseSpec responseSpec;
39
40     @Spy
41     private MatriculaRegistradaEventFactory matriculaRegistradaEventFactory = new MatriculaRegistradaEventFactory();
42
43     @InjectMocks private MatriculaServiceImpl matriculaService;
44
45     private Estudiante estudiante;
46     private Apoderado apoderado;
47
48     @BeforeEach
49     void setUp() {
50         apoderado = new Apoderado();
51         apoderado.setId(1L);
52         apoderado.setNombre("Juan Perez");
53         apoderado.setRut("12.345.678-9");
54         apoderado.setUsuarioId(20L);
55
56         estudiante = new Estudiante();
57         estudiante.setId(1L);
58         estudiante.setNombre("Diego Perez");
59         estudiante.setRut("23.456.789-0");
60         estudiante.setApoderado(apoderado);
61         estudiante.setEstado("Activo");
62         estudiante.setUsuarioId(10L);
63
64         lenient().when(webClient.get()).thenReturn(requestHeadersUriSpec);
65         lenient().when(requestHeadersUriSpec.uri(anyString())).thenReturn(requestHeadersSpec);
66         lenient().when(requestHeadersSpec.retrieve()).thenReturn(responseSpec);
67     }
68 }
```

setUp

- **Objetivo:** Inicializar el entorno de pruebas, configurar los mocks e instanciar las entidades base para los tests de `MatriculaServiceImpl`.
- **Descripción:** Instancia y llena datos de prueba para un `Apoderado` y un `Estudiante` (ID, RUT, nombres). Además, utiliza `lenient().when()` para preconfigurar el comportamiento encadenado del mock de `WebClient` (`get -> uri -> retrieve`), simulando la preparación de llamadas HTTP externas.

```

69      @Test
70      void listarTodosEstudiantes_DebeRetornarLista() {
71          when(estudianteRepository.findAll()).thenReturn(Arrays.asList(estudiante));
72
73          List<Estudiante> resultado = matriculaService.listarTodosEstudiantes();
74
75          assertFalse(resultado.isEmpty());
76          assertEquals(1, resultado.size());
77          verify(estudianteRepository, times(1)).findAll();
78      }
79
80
81      @Test
82      void listarTodosEstudiantes_CuandoHayError_DebeRetornarListaVacía() {
83          when(estudianteRepository.findAll()).thenReturn(new RuntimeException("Error de conexión"));
84
85          List<Estudiante> resultado = matriculaService.listarTodosEstudiantes();
86
87          assertTrue(resultado.isEmpty());
88      }

```

listarTodosEstudiantes_DebeRetornarLista

- **Objetivo:** Validar que el sistema devuelva correctamente la lista de alumnos registrados en el sistema.
- **Descripción:** Se configura el mock del repositorio para retornar una lista que contiene al estudiante de prueba. Se ejecuta `listarTodosEstudiantes()`, se confirma mediante `assertFalse` y `assertEquals` que el resultado no está vacío y posee un tamaño igual a uno, y se verifica que `findAll()` fue llamado una vez.

listarTodosEstudiantes_CuandoHayError_DebeRetornarListaVacía

- **Objetivo:** Validar que un fallo de conexión al listar alumnos retorne una lista vacía en lugar de romper la ejecución.
- **Descripción:** Configura el repositorio para lanzar una `RuntimeException` al llamar a `findAll()`. Ejecuta el método del servicio y verifica mediante `assertTrue`

que el resultado obtenido sea una lista vacía.

```
89      @Test
90      void buscarPorId_CuandoExiste_DebeRetornarEstudiante() {
91          when(estudianteRepository.findById(1L)).thenReturn(Optional.of(estudiante));
92
93          Estudiante resultado = matriculaService.buscarPorId(1L);
94
95          assertNotNull(resultado);
96          assertEquals(1L, resultado.getId());
97      }
98  }
```

buscarPorId_CuandoExiste_DebeRetornarEstudiante

- **Objetivo:** Validar la búsqueda exitosa de un estudiante mediante su ID cuando está registrado en el sistema.
- **Descripción:** Configura el repositorio para devolver al estudiante de prueba envuelto en un `Optional.of()` al buscar el ID `1L`. Ejecuta `buscarPorId(1L)`, confirmando mediante `assertNotNull` y `assertEquals` que el resultado no es nulo y contiene el identificador correcto.

```
99      @Test
100     void buscarPorId_CuandoNoExiste_DebeRetornarNull() {
101         when(estudianteRepository.findById(99L)).thenReturn(Optional.empty());
102
103         Estudiante resultado = matriculaService.buscarPorId(99L);
104
105         assertNull(resultado);
106     }
107 }
```

buscarPorId_CuandoNoExiste_DebeRetornarNull

- **Objetivo:** Validar que el sistema maneje correctamente la ausencia de un estudiante, retornando un valor nulo si el ID no existe.
- **Descripción:** Configura el repositorio para devolver un contenedor vacío (`Optional.empty()`) al buscar el ID `99L`. Ejecuta `buscarPorId(99L)` y verifica mediante `assertNull` que el resultado obtenido sea nulo.

```

108  @Test
109  void buscarEstudiantePorRut_CuandoExiste_DebeRetornarEstudiante() {
110      when(estudianteRepository.findByRut("23.456.789-0")).thenReturn(Optional.of(estudiante));
111
112      Estudiante resultado = matriculaService.buscarEstudiantePorRut("23.456.789-0");
113
114      assertNotNull(resultado);
115      assertEquals("23.456.789-0", resultado.getRut());
116  }
117

```

buscarEstudiantePorRut_CuandoExiste_DebeRetornarEstudiante

- **Objetivo:** Validar la localización exitosa de un alumno utilizando su RUT cuando está registrado en el sistema.
- **Descripción:** Se configura el repositorio para retornar al estudiante de prueba envuelto en un `Optional.of()` al buscar el RUT "23.456.789-0". El test ejecuta el método del servicio, confirmando con `assertNotNull` y `assertEquals` que el resultado es válido y mantiene el RUT consultado.

```

118  @Test
119  void registrarMatriculaCompleta_CuandoApoderadoEsNuevo_DebeGuardarAmbos() {
120      // Mock role checking
121      when(responseSpec.bodyToMono(Map.class))
122          .thenReturn(Mono.just(Map.of("rol", "Alumno")))
123          .thenReturn(Mono.just(Map.of("rol", "Apoderado")));
124
125      // Simulamos que el apoderado NO existe en la BD
126      when(apoderadoRepository.findByRut(any())).thenReturn(Optional.empty());
127      when(apoderadoRepository.save(any())).thenReturn(apoderado);
128      when(estudianteRepository.save(any(Estudiante.class))).thenReturn(estudiante);
129
130      Estudiante resultado = matriculaService.registrarMatriculaCompleta(estudiante);
131
132      assertNotNull(resultado);
133      verify(apoderadoRepository, times(1)).save(any(Apoderado.class));
134      verify(estudianteRepository, times(1)).save(estudiante);
135  }
136

```

registrarMatriculaCompleta_CuandoApoderadoEsNuevo_DebeGuardarAmbos

- **Objetivo:** Validar el flujo de inscripción completa cuando el apoderado no existe previamente en el sistema, asegurando la persistencia de ambas entidades.
- **Descripción:** Configura las llamadas consecutivas de `WebClient` para simular la verificación exitosa de roles ("Alumno" y "Apoderado"). Define que el apoderado no existe en la base de datos (`Optional.empty()`) y mockea los métodos `save` de ambos repositorios. Tras ejecutar la acción, confirma que el estudiante devuelto no es nulo y verifica que se invoque exactamente una vez el guardado de cada entidad.

```

137
138 void registrarMatriculaCompleta_CuandoApoderadoYaExiste_DebeReutilizarlo() {
139     // Mock role checking
140     when(responseSpec.bodyToMono(Map.class))
141         .thenReturn(Mono.just(Map.of("rol", "Alumno")))
142         .thenReturn(Mono.just(Map.of("rol", "Apoderado")));
143
144     // Simulamos que el apoderado SI existe
145     when(apoderadoRepository.findByRut(apoderado.getRut())).thenReturn(Optional.of(apoderado));
146     when(apoderadoRepository.save(any(Apoderado.class))).thenReturn(apoderado);
147     when(estudianteRepository.save(any(Estudiante.class))).thenReturn(estudiante);
148
149     Estudiante resultado = matriculaService.registrarMatriculaCompleta(estudiante);
150
151     assertNotNull(resultado);
152     assertNotNull(resultado.getApoderado());
153     assertEquals(apoderado.getId(), resultado.getApoderado().getId());
154     // Verificamos que se llamó al save de apoderado 1 vez para actualizar campos
155     verify(apoderadoRepository, times(1)).save(any(Apoderado.class));
156     verify(estudianteRepository).save(estudiante);
157 }
158 }
159

```

registrarMatriculaCompleta_CuandoApoderadoYaExiste_DebeReutilizarlo

- **Objetivo:** Validar la reutilización y actualización de un apoderado ya existente durante el flujo de matrícula.
- **Descripción:** Mockea las llamadas consecutivas de `WebClient` para los roles y configura al repositorio para retornar un apoderado existente (`Optional.of()`). Ejecuta la acción, valida con `assertEquals` que se asocie el mismo ID de apoderado al alumno y verifica que se llame una vez al `save()` de cada entidad para actualizar campos.

Test Servicio-Asistencia

AsistenciaServiceImplTest


```

24  @ExtendWith(MockitoExtension.class)
25  public class AsistenciaServiceImplTest {
26
27      @Mock
28      private AsistenciaRepository asistenciaRepository;
29
30      @Mock
31      private RabbitTemplate rabbitTemplate;
32
33      @Spy
34      private AsistenciaRegistradaEventFactory asistenciaRegistradaEventFactory = new AsistenciaRegistradaEventFactory();
35
36      @InjectMocks
37      private AsistenciaServiceImpl asistenciaService;
38
39      private Asistencia asistencia;
40
41      @BeforeEach
42      void setUp() {
43          asistencia = new Asistencia();
44      }
45

```

setUp

- **Objetivo:** Inicializar el entorno de pruebas y configurar los mocks, spies y el estado inicial requeridos para los tests de `AsistenciaServiceImpl`.
- **Descripción:** Se prepara una instancia limpia del objeto `Asistencia` antes de la ejecución de cada test. Este método configura los mocks de `AsistenciaRepository` y `RabbitTemplate`, junto con un spy de `AsistenciaRegistradaEventFactory`, inyectándolos automáticamente en el servicio de asistencia.

```

46
47  @Test
48  void listarTodas_Exito() {
49      // Arrange
50      List<Asistencia> listaEsperada = Arrays.asList(new Asistencia(), new Asistencia());
51      when(asistenciaRepository.findAll()).thenReturn(listaEsperada);
52
53      // Act
54      List<Asistencia> resultado = asistenciaService.listarTodas();
55
56      // Assert
57      assertNotNull(resultado);
58      assertEquals(2, resultado.size());
59      verify(asistenciaRepository, times(1)).findAll();
60  }

```

listarTodas_Exito

- **Objetivo:** Verificar el listado correcto de todos los registros de asistencia del sistema.
- **Descripción:** Se simula que `asistenciaRepository.findAll()` retorna una lista con dos registros de asistencia. El test ejecuta `listarTodas()`, valida que el resultado

obtenido no sea nulo, que contenga exactamente dos elementos y comprueba que la consulta al repositorio se haya realizado una única vez.

```
61  @Test
62  void listarTodas_Excepcion() {
63      // Arrange
64      when(asistenciaRepository.findAll()).thenReturn(new RuntimeException("Error simulado de base de datos"));
65
66      // Act
67      List<Asistencia> resultado = asistenciaService.listarTodas();
68
69      // Assert
70      assertNotNull(resultado);
71      assertTrue(resultado.isEmpty());
72      verify(asistenciaRepository, times(1)).findAll();
73  }
74
```

listarTodas_Excepcion

- **Objetivo:** Validar que el sistema controle correctamente una falla en la base de datos al listar las asistencias, retornando una lista vacía en lugar de romper la ejecución.
- **Descripción:** Se simula que `asistenciaRepository.findAll()` lanza una excepción de tipo `RuntimeException`. El test ejecuta `listarTodas()`, valida que el resultado obtenido no sea nulo, corrobora mediante `isEmpty()` que la lista devuelta esté vacía y verifica que se haya llamado al repositorio exactamente una vez.

```
75  @Test
76  void guardar_Exito() {
77      // Arrange
78      when(asistenciaRepository.save(any(Asistencia.class))).thenReturn(asistencia);
79
80      // Act
81      Asistencia resultado = asistenciaService.guardar(asistencia);
82
83      // Assert
84      assertNotNull(resultado);
85      verify(asistenciaRepository, times(1)).save(asistencia);
86  }
87
```

guardar_Exito

- **Objetivo:** Validar que el sistema puede persistir exitosamente un nuevo registro de asistencia.
- **Descripción:** Se simula que `asistenciaRepository.save(...)` retorna la instancia de asistencia configurada. El test ejecuta `guardar(asistencia)`, valida que el objeto devuelto no sea nulo y verifica que la operación de guardado en el repositorio se invoque exactamente una vez.

```
88      @Test
89      void guardar_Excepcion() {
90          // Arrange
91          when(asistenciaRepository.save(any(Asistencia.class)))
92              .thenReturn(new RuntimeException("Error simulado al guardar"));
93
94          // Act
95          Asistencia resultado = asistenciaService.guardar(asistencia);
96
97          // Assert
98          assertNull(resultado);
99          verify(asistenciaRepository, times(1)).save(asistencia);
100      }
101  }
```

guardar_Excepcion

- **Objetivo:** Validar que el sistema maneje correctamente una falla de infraestructura al intentar persistir una asistencia, asegurando que devuelva un valor nulo ante el error.
- **Descripción:** Se simula que `asistenciaRepository.save(...)` lanza una excepción `RuntimeException`. El test ejecuta `guardar(asistencia)`, valida mediante `assertNull` que el resultado sea nulo debido al fallo y verifica que se haya intentado realizar la operación de guardado en el repositorio exactamente una vez.

```
102      @Test
103      void buscarPorId_Encontrado() {
104          // Arrange
105          Long id = 1L;
106          when(asistenciaRepository.findById(id)).thenReturn(Optional.of(asistencia));
107
108          // Act
109          Asistencia resultado = asistenciaService.buscarPorId(id);
110
111          // Assert
112          assertNotNull(resultado);
113          verify(asistenciaRepository, times(1)).findById(id);
114      }
115  }
```

buscarPorId_Encontrado

- **Objetivo:** Validar la búsqueda exitosa de un registro de asistencia cuando este existe en el sistema.
- **Descripción:** Se simula que `asistenciaRepository.findById(1L)` devuelve un objeto de asistencia válido envuelto en un `Optional`. El test ejecuta `buscarPorId(id)`, valida mediante `assertNotNull` que el resultado obtenido no sea nulo y verifica que la consulta al repositorio se realice exactamente una vez.

```
116  @Test
117  void buscarPorId_NoEncontrado() {
118      // Arrange
119      Long id = 1L;
120      when(asistenciaRepository.findById(id)).thenReturn(Optional.empty());
121
122      // Act
123      Asistencia resultado = asistenciaService.buscarPorId(id);
124
125      // Assert
126      assertNull(resultado);
127      verify(asistenciaRepository, times(1)).findById(id);
128  }
129
```

buscarPorId_NoEncontrado

- **Objetivo:** Validar que el sistema maneje correctamente la ausencia de un registro de asistencia, retornando un valor nulo si el ID no existe.
- **Descripción:** Se simula que `asistenciaRepository.findById(1L)` retorna un contenedor vacío (`Optional.empty()`). El test ejecuta `buscarPorId(id)`, valida mediante `assertNull` que el resultado sea nulo y verifica que se haya consultado al repositorio exactamente una vez.

```
130  @Test
131  void buscarPorId_Excepcion() {
132      // Arrange
133      Long id = 1L;
134      when(asistenciaRepository.findById(id)).thenThrow(new RuntimeException("Error simulado al buscar por ID"));
135
136      // Act
137      Asistencia resultado = asistenciaService.buscarPorId(id);
138
139      // Assert
140      assertNull(resultado);
141      verify(asistenciaRepository, times(1)).findById(id);
142  }
143
```

buscarPorId_Excepcion

- **Objetivo:** Validar el manejo de errores del sistema cuando ocurre una falla en la infraestructura de persistencia al buscar una asistencia por su ID.
- **Descripción:** Se simula que `asistenciaRepository.findById(1L)` lanza una excepción de tipo `RuntimeException`. El test ejecuta `buscarPorId(id)`, confirma con un `assertNull` que el método intercepta la falla devolviendo un valor nulo, y comprueba que el intento de consulta al repositorio se realice una única vez.

```
143
144     @Test
145     void eliminar_Exito() {
146         // Arrange
147         Long id = 1L;
148         doNothing().when(asistenciaRepository).deleteById(id);
149
150         // Act
151         asistenciaService.eliminar(id);
152
153         // Assert
154         verify(asistenciaRepository, times(1)).deleteById(id);
155     }
156
```

eliminar_Exito

- **Objetivo:** Validar que el sistema procese correctamente la eliminación de un registro de asistencia utilizando su identificador único.
- **Descripción:** En la etapa de preparación (**Arrange**), se configura el comportamiento del mock **asistenciaRepository** mediante **doNothing()** para que no realice ninguna acción al invocar la eliminación del ID **1L**. Al ejecutar la acción (**Act**) llamando a **asistenciaService.eliminar(id)**, el test verifica en la aserción (**Assert**) que el método **deleteById(id)** del repositorio sea invocado exactamente una vez.

```
157
158     @Test
159     void eliminar_Excepcion() {
160         // Arrange
161         Long id = 1L;
162         doThrow(new RuntimeException("Error simulado al eliminar")).when(asistenciaRepository).deleteById(id);
163
164         // Act & Assert (verificar que no propaga la excepción debido al try-catch)
165         assertDoesNotThrow(() -> asistenciaService.eliminar(id));
166         verify(asistenciaRepository, times(1)).deleteById(id);
167     }
168
```

eliminar_Excepcion

- **Objetivo:** Verificar que el método capture y controle fallas de persistencia sin propagar la excepción.
- **Descripción:** Se configura el repositorio para lanzar una **RuntimeException** al borrar el ID **1L**. Se ejecuta el método dentro de un **assertDoesNotThrow()** para asegurar que el **try-catch** interno maneje el error, y se verifica que **deleteById** se invoque una vez.

Test Servicio-Academico

AcademicoServiceImplTest

```

31 @ExtendWith(MockitoExtension.class)
32 public class AcademicoServiceImplTest {
33     @Mock private CursoRepository cursoRepository;
34     @Mock private MatriculaRepository matriculaRepository;
35     @Mock private PruebaRepository pruebaRepository;
36     @Mock private NotaRepository notaRepository;
37     @Mock private com.proyecto.ColegioBackend.repository.CursoAsignaturaRepository cursoAsignaturaRepository;
38     @Mock private RabbitTemplate rabbitTemplate;
39     @Spy private NotaGeneradaEventFactory notaGeneradaEventFactory = new NotaGeneradaEventFactory();
40     @InjectMocks private AcademicoServiceImpl academicoService;
41     private Curso curso;
42     private Matricula matricula;
43     private Prueba prueba;
44     private Nota nota;
45     @BeforeEach
46     void setUp() {
47         curso = Curso.builder()
48             .id(1L)
49             .nombre("1 A")
50             .codigo("CUR-1-A")
51             .descripcion("Primer Año A")
52             .profesorId(10L)
53             .build();
54         matricula = Matricula.builder()
55             .id(1L)
56             .curso(curso)
57             .usuarioId(5L)
58             .build();
59         prueba = Prueba.builder()
60             .id(1L)
61             .titulo("Control 1")
62             .descripcion("Prueba de Matemáticas")
63             .fecha(LocalDate.now())
64             .curso(curso)
65             .build();
66         nota = Nota.builder()
67             .id(1L)
68             .prueba(prueba)
69             .alumnoId(5L)
70             .valor(6.5)
71             .comentario("Buen desempeño")
72             .build();
73     }
74 }

```

setUp

- **Objetivo:** Inicializar el entorno de pruebas y configurar los datos simulados (mocks y builders) necesarios para los tests de `AcademicoServiceImpl`.
- **Descripción:** Se preparan los objetos de negocio utilizando el patrón *Builder* con datos de prueba predefinidos: un **Curso** ("Primer Año A"), una **Matricula**, una **Prueba** ("Control 1" de Matemáticas) y una **Nota** con valor 6.5 y comentario de "Buen desempeño". Estos datos sirven como estado inicial para simular el comportamiento de los repositorios inyectados en el servicio académico.

```

76
77     @Test
78     void listarCursos_Exito() {
79         when(cursoRepository.findAll()).thenReturn(Arrays.asList(curso));
80         List<Curso> resultado = academicoService.listarCursos();
81         assertNotNull(resultado);
82         assertEquals(1, resultado.size());
83         verify(cursoRepository, times(1)).findAll();
84     }
85

```

listarCursos_Exito

- **Objetivo:** Verificar el listado correcto de todos los cursos.
- **Descripción:** Se simula que `cursoRepository.findAll()` retorna una lista con un curso. El test valida que el resultado de `listarCursos()` no sea nulo, contenga exactamente un elemento y que se consulte el repositorio una sola vez.

```

86
87     @Test
88     void obtenerCursoPorId_Encontrado() {
89         when(cursoRepository.findById(1L)).thenReturn(Optional.of(curso));
90         Curso resultado = academicoService.obtenerCursoPorId(1L);
91         assertNotNull(resultado);
92         assertEquals("1 A", resultado.getNombre());
93         verify(cursoRepository, times(1)).findById(1L);
94     }
95

```

obtenerCursoPorId_Encontrado

- **Objetivo:** Verificar la búsqueda exitosa de un curso por su ID.
- **Descripción:** Se simula que `cursoRepository.findById(1L)` devuelve un curso. El test valida que el resultado no sea nulo, que el nombre del curso sea "1 A" y que la consulta al repositorio se ejecute una única vez.

```

95
96     @Test
97     void crearCurso_Exito() {
98         when(cursoRepository.save(any(Curso.class))).thenReturn(curso);
99         when(cursoAsignaturaRepository.save(any(com.proyecto.ColegioBackend.model.CursoAsignatura.class))).thenReturn(null);
100         Curso nuevoCurso = Curso.builder().nombre("2 B").build();
101         Curso resultado = academicoService.crearCurso(nuevoCurso);
102         assertNotNull(resultado);
103         verify(cursoRepository, times(1)).save(any(Curso.class));
104         verify(cursoAsignaturaRepository, times(4)).save(any(com.proyecto.ColegioBackend.model.CursoAsignatura.class));
105     }
106

```

crearCurso_Exito

- **Objetivo:** Validar el registro de un nuevo curso y sus asignaturas.
- **Descripción:** Se simula el guardado del curso y sus relaciones en sus respectivos repositorios. Al registrar el curso "2 B", se verifica que el resultado no sea nulo, que el curso se salve una vez y sus asignaturas asociadas cuatro veces.

```
106 @Test
107 void asignarProfesor_Exito() {
108     when(cursoRepository.findById(1L)).thenReturn(Optional.of(curso));
109     when(cursoRepository.save(any(Curso.class))).thenReturn(curso);
110
111     Curso resultado = academicoService.asignarProfesor(1L, 20L);
112     assertNotNull(resultado);
113     verify(cursoRepository, times(1)).findById(1L);
114     verify(cursoRepository, times(1)).save(curso);
115 }
116
```

asignarProfesor_Exito

- **Objetivo:** Validar la asignación de un profesor a un curso.
- **Descripción:** Se simula la búsqueda del curso 1L y el guardado de su actualización. El test ejecuta `asignarProfesor(1L, 20L)` y verifica que el resultado no sea nulo, realizando exactamente una búsqueda y un guardado en el repositorio.

```
118
119 @Test
120 void matricularAlumno_CursoExisteYNoMatriculado() {
121     when(cursoRepository.findById(1L)).thenReturn(Optional.of(curso));
122     when(matriculaRepository.findByCursoIdAndUsuarioId(1L, 5L)).thenReturn(Optional.empty());
123     when(matriculaRepository.save(any(Matricula.class))).thenReturn(matricula);
124
125     Matricula resultado = academicoService.matricularAlumno(5L, 1L);
126     assertNotNull(resultado);
127     verify(cursoRepository, times(1)).findById(1L);
128     verify(matriculaRepository, times(1)).findByCursoIdAndUsuarioId(1L, 5L);
129     verify(matriculaRepository, times(1)).save(any(Matricula.class));
130 }
131
```

matricularAlumno_CursoExisteYNoMatriculado

- **Objetivo:** Validar la matrícula exitosa de un alumno cuando el curso existe y el estudiante no está inscrito previamente.
- **Descripción:** Se simula que el curso 1L existe, que la búsqueda de matrícula

previa retorna vacía (`Optional.empty()`) y se configura el guardado de la nueva matrícula. El test ejecuta `matricularAlumno(5L, 1L)`, valida que el resultado no sea nulo y verifica que se realice un único llamado a cada método de los repositorios (`findById`, `findByCursoIdAndUsuarioId` y `save`).

```
132  @Test
133  void matricularAlumno_YaMatriculado() {
134      when(cursoRepository.findById(1L)).thenReturn(Optional.of(curso));
135      when(matriculaRepository.findByCursoIdAndUsuarioId(1L, 5L)).thenReturn(Optional.of(matricula));
136
137      Matricula resultado = academicoService.matricularAlumno(5L, 1L);
138      assertNotNull(resultado);
139      assertEquals(1L, resultado.getId());
140      verify(matriculaRepository, never()).save(any(Matricula.class));
141  }
142
```

matricularAlumno_YaMatriculado

- **Objetivo:** Validar el comportamiento del sistema cuando se intenta matricular a un alumno que ya se encuentra inscrito en el curso.
- **Descripción:** Se simula que el curso `1L` existe y que la búsqueda de matrícula previa para el usuario `5L` ya retorna un registro existente. El test ejecuta `matricularAlumno(5L, 1L)`, valida que el resultado no sea nulo, verifica que el ID de la matrícula devuelta sea `1L` y comprueba mediante `never()` que **no** se intente guardar una nueva matrícula en el repositorio.

```
144
145  @Test
146  void crearPrueba_Exito() {
147      when(cursoRepository.findById(1L)).thenReturn(Optional.of(curso));
148      when(pruebaRepository.save(any(Prueba.class))).thenReturn(prueba);
149
150      Prueba resultado = academicoService.crearPrueba(1L, prueba);
151      assertNotNull(resultado);
152      verify(pruebaRepository, times(1)).save(prueba);
153  }
154
```

crearPrueba_Exito

- **Objetivo:** Validar la creación y asignación exitosa de una nueva evaluación (prueba) dentro de un curso existente.
- **Descripción:** Se simula que el curso `1L` existe en el repositorio y se configura el guardado de la evaluación. El test ejecuta `crearPrueba(1L, prueba)`, valida que el resultado obtenido no sea nulo y verifica mediante `verify` que la evaluación se

haya guardado en el repositorio exactamente una vez.

```
157
158 @Test
159 void registrarNota_NuevaNotaExito() {
160     when(pruebaRepository.findById(1L)).thenReturn(Optional.of(prueba));
161     when(notaRepository.findByPruebaIdAndAlumnoId(1L, 5L)).thenReturn(Optional.empty());
162     when(notaRepository.save(any(Nota.class))).thenReturn(nota);
163
164     Nota resultado = academicoService.registrarNota(1L, 5L, 6.5, "Muy bien");
165     assertNotNull(resultado);
166     verify(notaRepository, times(1)).save(any(Nota.class));
167 }
```

registrarNota_NuevaNotaExito

- **Objetivo:** Validar el registro exitoso de una calificación para un alumno en una prueba específica cuando no cuenta con una nota previa.
- **Descripción:** Se simula que la prueba 1L existe, que la búsqueda de una nota previa para el alumno 5L retorna vacía (`Optional.empty()`) y se configura el guardado de la nueva calificación. El test ejecuta `registrarNota(1L, 5L, 6.5, "Muy bien")`, valida que el resultado no sea nulo y verifica que se realice exactamente un guardado en el repositorio (`notaRepository.save`).

```
167
168 @Test
169 void registrarNota_ActualizarNotaExito() {
170     when(pruebaRepository.findById(1L)).thenReturn(Optional.of(prueba));
171     when(notaRepository.findByPruebaIdAndAlumnoId(1L, 5L)).thenReturn(Optional.of(nota));
172     when(notaRepository.save(any(Nota.class))).thenReturn(nota);
173
174     Nota resultado = academicoService.registrarNota(1L, 5L, 7.0, "Excelente");
175     assertNotNull(resultado);
176     assertEquals(7.0, resultado.getValor());
177     assertEquals("Excelente", resultado.getComentario());
178     verify(notaRepository, times(1)).save(nota);
179 }
180
```

registrarNota_ActualizarNotaExito

- **Objetivo:** Validar la actualización exitosa de una calificación existente cuando el alumno ya cuenta con una nota previa en la prueba.
- **Descripción:** Se simula que la prueba 1L existe y que la búsqueda de la nota ya retorna un registro previo. El test ejecuta `registrarNota(1L, 5L, 7.0, "Excelente")`, valida que el resultado no sea nulo, corrobora que el valor y comentario se hayan actualizado correctamente, y verifica que se realice exactamente un guardado en el repositorio.

```
181
182 @Test
183 void registrarNota_ValorInvalidoLanzaExcepcion() {
184     assertThrows(IllegalArgumentException.class, () -> {
185         academicoService.registrarNota(1L, 5L, 7.5, "Invalida");
186     });
187     assertThrows(IllegalArgumentException.class, () -> {
188         academicoService.registrarNota(1L, 5L, 0.9, "Invalida");
189     });
190     verify(notaRepository, never()).save(any(Nota.class));
191 }
192
```

registrarNota_ValorInvalidoLanzaExcepcion

- **Objetivo:** Validar que el sistema impida el registro de calificaciones fuera del rango permitido, lanzando la excepción correspondiente.
- **Descripción:** El test utiliza `assertThrows` para verificar que se lanza una excepción `IllegalArgumentException` al intentar registrar notas inválidas (un `7.5` o un `0.9`). Además, se comprueba mediante `never()` que el repositorio **no** realiza ninguna acción de guardado en la base de datos debido a estas fallas de validación.

9. Branching

Backend

Branch	Updated	Check status	Behind	Ahead	Pull request
features/docker-dilan-micolta 	 yesterday		3	1	 #9  ...
features/servicio-academico-vicente-manriquez 	 4 days ago		4	1	 ...
feature/servicio-reportes-davier-ramos 	 last week		4	0	 #8  ...
features/rabbitmq-circuit-breaker-dilan-micolta 	 last week		20	1	 ...
fix/gestion-curso-vicente-manriquez 	 last week		5	1	 #7  ...
features/webclient-vicente-manriquez 	 3 weeks ago		5	1	 #6  ...
feature/pruebas-unitarias-dilan-micolta 	 last month		20	2	 ...
feature/try-catches-dilan-micolta 	 last month		20	1	 ...
features/servicio-asistencia-3.0-vicente-manriquez 	 last month		16	1	 #5  ...
feature/servicio-matricula-dilan-micolta 	 last month		20	1	 ...
features/test-unitarios-servicio-asistencia-vicente-manriquez 	 2 months ago		19	1	 #4  ...
feature/servicio-usuario-test-davier-ramos 	 2 months ago		20	2	 ...
feature/servicio-usuario-davier-ramos 	 2 months ago		20	1	 ...
features/servicio-asistencia-2.0-vicente-manriquez 	 2 months ago		19	1	 #3  ...
fix/servicio-asistencia-vicente-manriquez 	 2 months ago		19	1	 #2  ...
fix/Subida-Backend-Beta-01-Vicente-Manriquez 	 2 months ago		20	1	 ...

Ramas (Branch): Tareas específicas de desarrollo (**features** o **fix**) divididas por programador (Vicente, Dilan, Davier).

Sincronización (Behind / Ahead): Qué tan desactualizada está cada rama respecto a la principal y cuántos cambios nuevos tiene.

Pull Requests: El estado de las revisiones de código listas para fusionarse (íconos verdes con #).

Actividad (Updated): Cuándo se realizó la última modificación.

Frontend

Q Search branches...					
Branch	Updated	Check status	Behind	Ahead	Pull request
Feature/frontend-final-davier-ramos	yesterday		5	1	
Features/servicio-academico-vicente-manriquez	4 days ago		1	1	#3
Features/WebClient-vicente-manriquez	3 weeks ago		1	1	#2
Feature/frontend-final-Davier-Ramos	last month		4	0	
Feature/validaciones-dilan-micolta	last month		5	1	
Features/frontend-2.0-vicente-manriquez	last month		5	4	#1
Feature/colegio-frontend-1.1-davier-ramos	last month		5	1	
Feature/colegio-frontend-davier-ramos	2 months ago		5	1	

Ramas (Branch): Muestra el desarrollo de la interfaz (como **frontend-final**, **validaciones** o **colegio-frontend**), trabajado por el mismo equipo (Davier, Vicente y Dilan).

Estado de Pull Requests: Tiene menos solicitudes activas en comparación con el backend; solo se ven tres *Pull Requests* creados (**#3**, **#2** y **#1**).

Sincronización y Actividad: Al igual que el anterior, indica qué tan desactualizada está cada tarea respecto a la rama principal (**Behind/Ahead**) y cuándo fue su último cambio (desde "ayer" hasta "hace 2 meses").

10. Sonarcube

```
[INFO] -----
[INFO] Reactor Summary:
[INFO]
[INFO] apigateway 0.0.1-SNAPSHOT ..... SUCCESS [ 17.828 s]
[INFO] colegio 0.0.1-SNAPSHOT ..... SUCCESS [ 16.844 s]
[INFO] Servicio Asistencia 1.0.0 ..... SUCCESS [ 6.338 s]
[INFO] servicio-matricula 0.0.1-SNAPSHOT ..... SUCCESS [ 21.866 s]
[INFO] Servicio Academico 1.0.0 ..... SUCCESS [ 11.919 s]
[INFO] Servicio Reportes 1.0.0 ..... SUCCESS [ 6.542 s]
[INFO] Servicio Mensajeria 1.0.0 ..... SUCCESS [ 6.105 s]
[INFO] Colegio System - Parent POM 1.0.0 ..... SUCCESS [ 26.799 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:55 min
[INFO] Finished at: 2026-06-14T19:44:21-04:00
[INFO] -----
PS C:\Users\vicho\Downloads\proyecto\proyecto\colegio>
```

SonarQube es una plataforma de **Análisis Estático de Código (SCA)** de estándar industrial. Su función principal es la "Inspección Continua", lo que significa que escanea el código fuente sin ejecutarlo para identificar problemas de calidad, fallos de seguridad y deudas técnicas.

Cuando SonarQube marca un proyecto como **"Passed"**, significa que el código ha superado el **Quality Gate** (Umbral de Calidad) configurado. El Quality Gate es un conjunto de condiciones mínimas que el software debe cumplir antes de ser considerado "listo para producción".

Tus resultados indican que tus microservicios cumplen satisfactoriamente en los siguientes cinco pilares:

- **Reliability (Confiabilidad - Bugs):** Se detectaron cero o muy pocos errores de lógica que pudieran causar un comportamiento inesperado o el cierre del servicio. Esto garantiza la estabilidad de la plataforma escolar.
- **Security (Seguridad - Vulnerabilidades):** El análisis confirma que no existen brechas de seguridad críticas (como inyecciones de código o gestión insegura de datos). Es vital dado que el sistema maneja información sensible de estudiantes y credenciales de acceso (JWT).
- **Maintainability (Mantenibilidad - Code Smells):** Indica que el código es limpio y fácil de leer. Una baja densidad de "Code Smells" significa que la **Deuda Técnica** es mínima, lo que facilitará que otros desarrolladores (o tú mismo en el futuro) puedan actualizar el sistema sin introducir errores fácilmente.
- **Coverage (Cobertura de Pruebas):** Para obtener el éxito, SonarQube suele validar que una parte significativa del código (generalmente >80%) esté cubierta por pruebas unitarias (como las que tienes en `ReporteCompletoServiceTest.java`). Esto asegura que el sistema está protegido contra regresiones.
- **Duplications (Duplicidad):** El código sigue el principio DRY (*Don't Repeat Yourself*). Hay poca o nula repetición de lógica, lo que hace al sistema más eficiente y fácil de corregir.

Con el uso de sonarqube garantizamos:

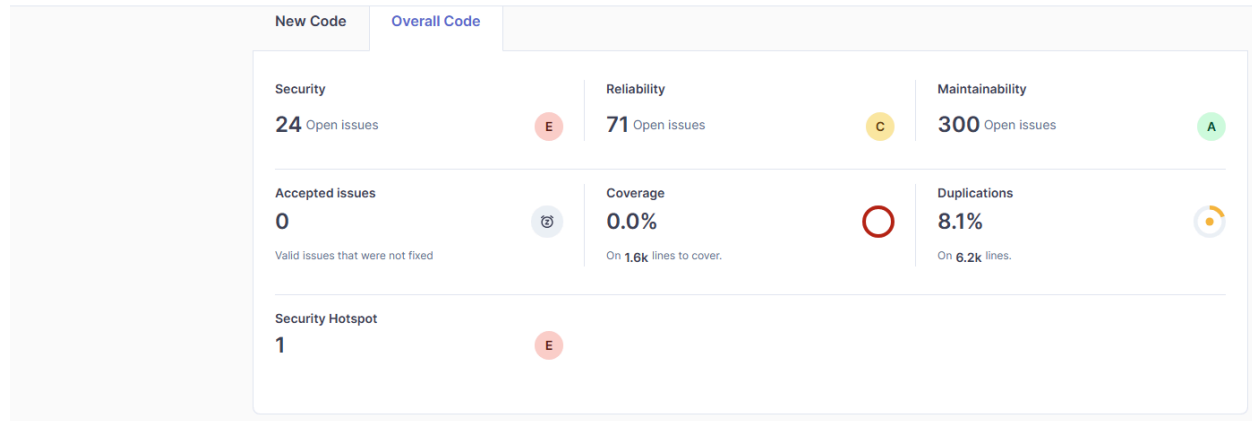
1. **Reducción de Riesgos:** Menos errores en producción.
2. **Escalabilidad Técnica:** Un código limpio permite añadir nuevos microservicios (como el de mensajería o reportes) sin degradar la calidad global.
3. **Profesionalismo:** El cumplimiento de métricas de calidad técnica es un diferenciador clave que eleva el estándar del proyecto de un nivel académico a uno profesional.

11.Evaluación de seguridad

Colegio System - Parent POM / Overview

Overview

4.9k Lines of Code · Version 1.0.0 · Last analysis 1 day ago



En nuestro sonnar podemos ver los siguientes resultados y analisis:

Prioridad Crítica (Acción Inmediata)

- **Security (Calificación E - 24 Problemas / 1 Hotspot):** Es la mayor vulnerabilidad del sistema. La nota **E** implica brechas críticas que exponen el software a posibles accesos no autorizados o fallos de seguridad graves. El *Security Hotspot* requiere auditoría manual inmediata.
- **Coverage (0.0% de Cobertura - 1.6K líneas sin testear):** El sistema carece por completo de pruebas unitarias o automatizadas. Al no haber cobertura, cualquier modificación en los microservicios puede generar **regresiones** (errores ocultos que rompen partes del sistema que antes funcionaban).

Prioridad Media (Refactorización y Estabilización)

- **Reliability (Calificación C - 71 Bugs):** Existen errores lógicos funcionales de gravedad media. Si bien el sistema compila, estos bugs comprometen la estabilidad operativa de la plataforma escolar en tiempo de ejecución.
- **Duplications (8.1% de Código Duplicado):** Se infringe parcialmente el principio **DRY** (*Don't Repeat Yourself*). Hay fragmentos idénticos copiados y pegados en la densidad de las 6.2K líneas, lo que eleva innecesariamente el esfuerzo de mantenimiento futuro.

Estado Óptimo

- **Maintainability (Calificación A - 300 Code Smells):** A pesar del volumen de malas prácticas menores, la **Deuda Técnica** (tiempo estimado para corregirlas) es mínima respecto al tamaño del código. La arquitectura base es legible, limpia y fácil de mantener.

12. swagger



Descripción de la Herramienta

- **Plataforma:** Se presenta la interfaz gráfica de **Swagger UI**, generada bajo la especificación estándar **OpenAPI 3.0**.
- **Propósito:** Se utiliza para la documentación interactiva, el diseño y las pruebas de los diferentes *endpoints* que componen la API del sistema.
- **Entorno:** En esta sección particular, se exhiben de forma estructurada los servicios del backend expuestos localmente en el puerto **8081** (**localhost:8081**).

Componentes y Controladores Expuestos

La interfaz organiza los métodos de comunicación en tres controladores principales, detallando sus respectivas operaciones e instrucciones HTTP:

- **auth-controller (Gestión de Autenticación):**
 - Encargado de los procesos de registro e inicios de sesión en la plataforma.
 - Utiliza el método **POST** para el envío seguro de credenciales (**/api/auth/register** y **/api/auth/login**).
- **mensajeria-controller (Módulo de Mensajería):**
 - Enfocado en la lógica de envío de alertas y comunicaciones distribuidas dentro del ecosistema.
 - Utiliza el método **POST** para procesar las solicitudes (**/api/mensajes/enviar**).
- **usuario-controller (Administración de Usuarios):**
 - Provee las operaciones **CRUD** fundamentales (Crear, Leer, Actualizar y Eliminar) sobre la entidad de usuarios del sistema.
 - Emplea de manera estructurada los verbos HTTP estándar: **GET**

(lectura/consulta), **POST** (creación), **PUT** (modificación por ID) y **DELETE** (remoción por ID).

13. Bibliografía

- Fowler, M. (2018). *Microservices Patterns*. Manning Publications.
- Newman, S. (2021). *Building Microservices*. O'Reilly Media.
- Oracle. (2023). *Java Persistence API Documentation*.
- React Documentation. (2024). *React.dev*.
- Richardson, C. (2020). *Microservices Patterns*. Manning Publications.